Un serviciu, multe calculatoare

Sisteme distribuite: colaborare între calculatoare

SCAN / LEARN

IDEEA CENTRALĂ

Un sistem distribuit este format din calculatoare care colaborează prin rețea pentru a oferi un serviciu. Mesajele pot întârzia sau se pot pierde.

URMĂREȘTE SISTEMUL

01 — O CERERE

Un utilizator solicită o pagină sau un rezultat.

02 — MAI MULTE NODURI

Calculatoarele execută părți ale serviciului.

03 — MESAJE

Nodurile își transmit informații prin rețea.

04 — UN RĂSPUNS

Serviciul combină rezultatele și răspunde.

UTILIZATOR → SERVICIU → NODURI → UTILIZATOR

CONFUZIE FRECVENTĂ

Mai multe calculatoare înseamnă automat viteză mai mare.

MAI CORECT

Rețeaua, datele comune și coordonarea pot încetini totul.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum oferă mai multe calculatoare un serviciu?

DACĂ EȘUEAZĂ: O parte întârziată poate bloca răspunsul.

ÎN REALITATE: Magazine online, căutare, bănci și AI.

EXPLORARE INDIVIDUALĂ

Desenează două servere: ce se întâmplă dacă unul pierde rețeaua?

Ce se întâmplă când cade un server?

Defecțiuni, replici și recuperare

SCAN / LEARN

IDEEA CENTRALĂ

Defecțiunile sunt normale: componentele se strică, rețelele se întrerup, programele se blochează. Un serviciu rezilient detectează și ocolește problemele.

URMĂREȘTE SISTEMUL

01

VERIFICARE

Sistemul testează dacă serverul răspunde.

02

DEFECȚIUNE

Un server sau o legătură devine indisponibilă.

03

REDIRECȚIONARE

O replică sănătoasă preia cererile noi.

04

RECUPERARE

Componenta este reparată și verificată.

VERIFICĂ → DETECTEAZĂ → MUTĂ → REPARĂ

CONFUZIE FRECVENTĂ

Dacă există un server de rezervă, nimic nu poate eșua.

MAI CORECT

Alimentarea, stocarea ori configurația comună pot ceda.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum supraviețuiește un serviciu unei defecțiuni?

DACĂ EȘUEAZĂ: Lipsa replicilor poate opri serviciul.

ÎN REALITATE: Aplicații cloud și plăți online.

EXPLORARE INDIVIDUALĂ

Desenează două servere cu o bază de date comună. Ce poate ceda?

Trei copii: care este corectă?

Replicare și consistența datelor

SCAN / LEARN

IDEEA CENTRALĂ

Replicarea păstrează copii ale datelor în locuri diferite. Uneori copiile nu coincid temporar; sistemul are nevoie de reguli pentru actualizări și conflicte.

URMĂREȘTE SISTEMUL

01 ORIGINAL

Un client își modifică adresa.

02 PROPAGARE

Modificarea este transmisă către replici.

03 ÎNTÂRZIERE

O replică păstrează încă adresa veche.

04 RECONCILIERE

Regula de consistență stabilește rezultatul.

MODIFICĂ → COPIAZĂ → ÎNTÂRZIE → DECIDE

CONFUZIE FRECVENTĂ

Trei copii sunt întotdeauna identice în orice moment.

MAI CORECT

Unele arhitecturi acceptă diferențe scurte pentru disponibilitate.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum rămân utile copiile după o actualizare?

DACĂ EȘUEAZĂ: Informațiile vechi produc decizii greșite.

ÎN REALITATE: Baze de date cloud și servicii globale.

EXPLORARE INDIVIDUALĂ

Scrie două modificări diferite pe foi. Care ar trebui păstrată?

Două programe, un singur număr

Condiții de cursă și date partajate

SCAN / LEARN

IDEEA CENTRALĂ

O condiție de cursă apare când rezultatul depinde de ordinea operațiilor. Fără coordonare, două actualizări corecte separat se pot suprascrie.

URMĂREȘTE SISTEMUL

01 START

Soldul comun este 10.

02 CITIRE

Două programe citesc fiecare valoarea 10.

03 CALCUL

Fiecare calculează separat 10 + 1.

04 SUPRASCRIERE

Ambele scriu 11, deși au fost două modificări.

CITEȘTE 10 → DOUĂ MODIFICĂRI → SCRIE 11

CONFUZIE FRECVENTĂ

Dacă fiecare program este corect, și rezultatul comun este.

MAI CORECT

Modificările comune cer ordine sigură sau operații atomice.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum evităm pierderea modificărilor simultane?

DACĂ EȘUEAZĂ: O actualizare dispare fără mesaj de eroare.

ÎN REALITATE: Contoare, stocuri și solduri.

EXPLORARE INDIVIDUALĂ

Simulează pe hârtie două citiri și două scrieri asupra lui 10.

Tranzacția: totul sau nimic

Modificări sigure ale bazei de date

SCAN / LEARN

IDEEA CENTRALĂ

O tranzacție grupează modificări legate într-o singură unitate. Dacă un pas obligatoriu eșuează, operația neterminată poate fi anulată.

URMĂREȘTE SISTEMUL

01 ÎNCEPUT

Un client transferă bani.

02 DEBIT

Se scade suma din primul cont.

03 CREDIT

Se adaugă suma în al doilea cont.

04 CONFIRMĂ / ANULEAZĂ

Salvează ambele modificări sau le anulează.

ÎNCEPE → MODIFICĂ A → MODIFICĂ B → CONFIRMĂ

CONFUZIE FRECVENTĂ

Două instrucțiuni consecutive nu pot fi separate de o eroare.

MAI CORECT

Avem nevoie de tranzacții și de reguli de izolare corecte.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum reușesc împreună modificările dependente?

DACĂ EȘUEAZĂ: Operațiile parțiale lasă datele incorecte.

ÎN REALITATE: Transferuri, comenzi și rezervări.

EXPLORARE INDIVIDUALĂ

Imaginează o rezervare în doi pași. Ce faci dacă pasul doi eșuează?

Dirijorul traficului

Distribuirea cererilor și blocajele

SCAN / LEARN

IDEEA CENTRALĂ

Un distribuitor de trafic direcționează cererile spre servere funcționale. Poate reduce supraîncărcarea, dar nu creează capacitate nelimitată.

URMĂREȘTE SISTEMUL

01 CERERI

Multe dispozitive accesează același serviciu.

02 DISTRIBUITOR

Punctul de intrare alege un server sănătos.

03 SERVERE

Mai multe mașini execută cererile.

04 FEEDBACK

Măsurătorile modifică direcționarea.

VIZITATORI → DISTRIBUITOR → SERVERE

CONFUZIE FRECVENTĂ

Distribuirea traficului accelerează orice bază de date.

MAI CORECT

Un blocaj comun rămâne până îi schimbăm capacitatea.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum împărțim traficul între servere?

DACĂ EȘUEAZĂ: Un server e sufocat, celelalte așteaptă.

ÎN REALITATE: Magazine, video și servicii AI.

EXPLORARE INDIVIDUALĂ

Distribuie 12 cereri numerotate la 3 servere, pe rând.

Un program într-un container

Ambalarea software-ului pentru rulare predictibilă

SCAN / LEARN

IDEEA CENTRALĂ

Un container împachetează aplicația și dependențele ei. Containerele folosesc în comun nucleul sistemului gazdă; nu sunt automat mașini virtuale separate.

URMĂREȘTE SISTEMUL

01 APLICAȚIE

Un program cere anumite biblioteci.

02 IMAGINE

Codul și dependențele sunt împachetate.

03 CONTAINER

Imaginea rulează ca proces izolat.

04 GAZDĂ

Calculatorul oferă nucleu și resurse comune.

COD + BIBLIOTECI → IMAGINE → CONTAINER

CONFUZIE FRECVENTĂ

Un container include propriul sistem de operare complet.

MAI CORECT

Izolarea are limite; nucleul gazdei rămâne comun.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum rulează programul previzibil și în alt loc?

DACĂ EȘUEAZĂ: Dependențele sau permisiunile îl blochează.

ÎN REALITATE: Servicii, dezvoltare și publicare.

EXPLORARE INDIVIDUALĂ

Notează ce îi trebuie unei aplicații în afară de cod.

Cine conduce toate containerele?

Kubernetes: orchestrarea containerelor

SCAN / LEARN

08 / 12 • GHID VIZUAL

IDEEA CENTRALĂ

Kubernetes coordonează containere distribuite între calculatoare. Observă configurația dorită și încearcă să păstreze serviciile reale aproape de ea.

URMĂREȘTE SISTEMUL

01 STARE DORITĂ

Un serviciu ar trebui să aibă trei copii active.

02 PLANIFICATOR

Kubernetes distribuie sarcini pe mașini.

03 OBSERVĂ

Verifică ce copii funcționează corect.

04 CORECTEAZĂ

Repornește sau înlocuiește copiile lipsă.

STARE DORITĂ → ALOCARE → VERIFICĂ → REPARĂ

CONFUZIE FRECVENTĂ

Kubernetes garantează că orice aplicație este corectă.

MAI CORECT

Gestionează rularea, nu repară logica greșită a aplicației.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum coordonăm multe containere simultan?

DACĂ EȘUEAZĂ: Setări greșite pot declanșa reporniri.

ÎN REALITATE: Clustere cloud și aplicații mari.

EXPLORARE INDIVIDUALĂ

Desenează 3 mașini și așază 5 sarcini cu spațiu limitat.

Cache comun pentru multe servere

Memorare temporară și date învechite

SCAN / LEARN

IDEEA CENTRALĂ

Un cache păstrează temporar rezultate reutilizabile. Un cache partajat ajută mai multe servere, dar informațiile vechi sau greșite pot propaga erori.

URMĂREȘTE SISTEMUL

01 CERERE

O aplicație solicită un rezultat salvat.

02 CĂUTARE

Cache-ul verifică dacă are o copie validă.

03 GĂSIT / LIPSĂ

Refolosește copia sau întreabă sursa.

04 REÎNNOIRE

O regulă expiră ori actualizează datele.

CERERE → CACHE → REFOLOSEȘTE SAU CERE → REÎNNOIEȘTE

CONFUZIE FRECVENTĂ

Păstrarea tuturor răspunsurilor e mereu sigură și ieftină.

MAI CORECT

Prospețimea, confidențialitatea și costul decid.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum evităm recalcularea acelorași rezultate?

DACĂ EȘUEAZĂ: Un preț vechi poate induce clientul în eroare.

ÎN REALITATE: Site-uri, baze de date și fluxuri AI.

EXPLORARE INDIVIDUALĂ

Scrie 3 informații schimbătoare. Pe care le refolosești mâine?

De ce este lentă rețeaua?

Latență, lățime de bandă și așteptare

SCAN / LEARN

IDEEA CENTRALĂ

Latența este întârzierea până când ajung datele. Lățimea de bandă arată câtă informație poate circula într-o secundă. Mai multă bandă nu elimină orice așteptare.

URMĂREȘTE SISTEMUL

01 CERERE

Pleacă un mesaj mic de pe dispozitiv.

02 DRUM

Rețeaua adaugă întârzieri de transmitere.

03 AȘTEPTARE

Serverele ocupate țin cererile în coadă.

04 RĂSPUNS

Calculul și drumul înapoi consumă timp.

LATENȚĂ = DRUM + AȘTEPTARE + PROCESARE

CONFUZIE FRECVENTĂ

Un abonament rapid face instantanee toate serviciile.

MAI CORECT

Distanța, aglomerația și procesarea tot cer timp.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: De ce aceeași cerere durează diferit?

DACĂ EȘUEAZĂ: Întârzierile repetate se acumulează.

ÎN REALITATE: Jocuri, apeluri video și AI la distanță.

EXPLORARE INDIVIDUALĂ

Compară descărcarea unui fișier mare cu un răspuns mic, dar lent.

Cum văd inginerii invizibilul?

Jurnale, metrici și urmărirea cererilor

SCAN / LEARN

IDEEA CENTRALĂ

Observabilitatea înseamnă înțelegerea sistemului prin semnalele sale. Jurnalele notează evenimente, metricile arată valori în timp, iar traseele urmăresc cererile.

URMĂREȘTE SISTEMUL

01 JURNAL

Un mesaj notează ce s-a întâmplat și când.

02 METRICĂ

Un grafic arată erori sau timpi de răspuns.

03 TRASEU

O cerere e urmărită prin mai multe servicii.

04 DIAGNOSTIC

Semnalele împreună sugerează cauza.

EVENIMENTE + NUMERE + TRASEE → DIAGNOSTIC

CONFUZIE FRECVENTĂ

Un singur grafic explică întotdeauna întreaga problemă.

MAI CORECT

Graficul arată simptomul; traseul și jurnalul caută cauza.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum investigăm o încetinire invizibilă?

DACĂ EȘUEAZĂ: Fără semnale, căutăm cauza prin presupuneri.

ÎN REALITATE: Fiabilitatea serviciilor și AI.

EXPLORARE INDIVIDUALĂ

Imaginează un site lent. Ce jurnal, metrică și traseu ai cere?

Mai rapid, mai ieftin sau mai sigur?

Compromisurile proiectării sistemelor

SCAN / LEARN

IDEEA CENTRALĂ

Orice proiect echilibrează costul, viteza și fiabilitatea. Optimizarea începe cu un obiectiv măsurabil, apoi verifică beneficiile și efectele secundare.

URMĂREȘTE SISTEMUL

01 MĂSOARĂ

Notează viteza, erorile și costul resurselor.

02 ALEGE

Identifică un blocaj care merită rezolvat.

03 MODIFICĂ

Testează cache, alocări sau rute mai bune.

04 COMPARĂ

Măsoară rezultatul nou și riscurile lui.

MĂSOARĂ → MODIFICĂ → MĂSOARĂ DIN NOU → DECIDE

CONFUZIE FRECVENTĂ

Soluția cea mai ieftină este mereu cea mai eficientă.

MAI CORECT

O defecțiune ieftină poate produce pierderi mari.

TREI ÎNTREBĂRI DE REȚINUT

PROBLEMA: Cum economisim resurse fără riscuri inutile?

DACĂ EȘUEAZĂ: Un indicator se îmbunătățește, altul scade.

ÎN REALITATE: Costuri cloud, inferență AI și streaming.

EXPLORARE INDIVIDUALĂ

Compară două variante: una mai rapidă, dar dublu de scumpă.